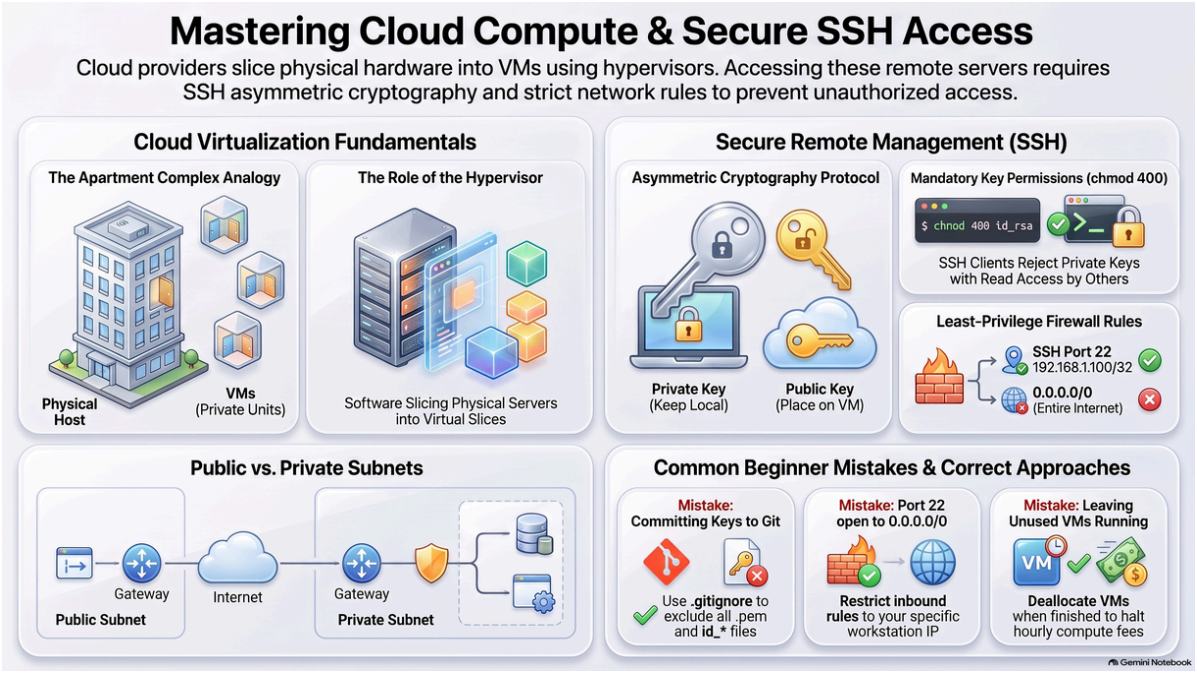
Takuya's Lesson 77: *Tuesday, September 29, 2026, 1400–1515*

Cloud Engineering Lesson 12

Phase 2: Cloud Foundations

Virtual Machines (Azure VM, AWS EC2, and Secure SSH Operations)



Learning Objectives

By the end of today's lesson, you will be able to:

1. Explain how hypervisors and cloud virtualization deliver scalable on-demand compute instances (Azure Virtual Machines and AWS EC2).

2. Generate, manage, and secure asymmetric cryptographic key pairs (ssh-keygen, private key permissions via chmod 400).

3. Connect securely to remote Linux cloud servers over port 22 using SSH.

4. Configure Network Security Groups (NSGs) / Security Groups to enforce least-privilege inbound traffic rules.

5. Observe live demonstrations of SSH connectivity audits, VM metadata queries, and remote command execution run directly in VS Code.

6. Provision an Ubuntu Server virtual machine in the cloud and configure web server access.

Section 1: Warm-Up Review

Answer using complete sentences (maximum 2 sentences per question).

1. **Why do cloud providers (like AWS and Azure) reserve 5 IP addresses in every subnet?**

2. **What is the operational difference between a Public Subnet and a Private Subnet?**

3. **What is the role of an Internet Gateway (IGW) in a cloud network?**

4. **Why is using overlapping CIDR blocks (e.g., two networks using 10.0.0.0/16) problematic?**

5. **Why should backend databases never be placed inside a public subnet with an open gateway route?**

Section 2: Why Cloud Compute & Virtual Machines Matter

In modern cloud computing, compute instances are the workhorses that execute backend application code, manage database engines, run container hosts, and process batch automation jobs. In on-premises infrastructure, standing up a server required waiting weeks for physical hardware to arrive, mounting it into a server rack, running Ethernet drops, installing an operating system from a flash drive, and configuring BIOS firmware.

Cloud Virtual Machines (known as **Virtual Machines** in Azure and **Elastic Compute Cloud / EC2** in AWS) virtualize physical hardware. Cloud providers use hypervisors to split massive physical host computers into isolated, right-sized virtual slices. With Infrastructure as Code (IaC) or cloud consoles, an engineer can spin up an Ubuntu Linux server with 4 CPUs and 16 GB of RAM in under sixty seconds.

To understand how virtual machines and secure access work, consider an analogy involving a **high-security apartment complex**:

- **The Physical Server Host is the Entire Apartment Complex:** It provides the physical concrete foundation, central power grid, and structural water plumbing (hypervisor and physical data center).

- **The Virtual Machine (VM/EC2) is a Rented Apartment Unit:** You have your own private living room, furniture, and customized interior (CPU, RAM, storage, and guest operating system). You cannot see inside your neighbor's unit, and they cannot see inside yours.

- **SSH (Secure Shell) is Your Unique Metal Physical Key:** You do not enter using an insecure combination lock that anyone can guess. You carry a cryptographic private key that matches the lock cylinder on your specific apartment door.

- **The Network Security Group (NSG) is the Front Desk Security Guard:** The guard inspects the visitor log. If someone arrives on Port 22 (SSH) or Port 80 (HTTP) and their IP is on the approved list, the guard lets them pass; otherwise, they are turned away at the door.

Mastering Linux server provisioning, SSH authentication, and security group isolation is fundamental to managing cloud compute fleets safely and professionally.

Question 1: The Leaked Private Key Catastrophe

- **Scenario:** A developer generates an SSH key pair (id_rsa and id_rsa.pub) to manage an AWS EC2 instance. While creating a project repository, the developer accidentally commits the private key file (id_rsa) to a public GitHub repository. Within 15 minutes, automated bots scan the commit, pull the key, and gain root access to the server.

- **Question:** What is the technical difference between an SSH public key and a private key, and which one is safe to share or upload to a remote server?

Question 2: The Open Port 22 Ransomware Breach

- **Scenario:** An administrator creates a cloud virtual machine running Ubuntu Server. In the Network Security Group, the administrator creates an inbound rule allowing Port 22 (SSH) from 0.0.0.0/0 (Any IP on the internet). Two days later, server logs show hundreds of thousands of failed login attempts from global botnets brute-forcing root passwords.

- **Question:** How should Network Security Group rules for management ports (like SSH port 22 or RDP port 3389) be restricted in production?

Section 3: SSH Key Cryptography & Permissions

Technical Explanation

Cloud virtual machines use asymmetric cryptography for secure remote management:

- **The Key Pair:** Generated using ssh-keygen -t ed25519 or ssh-keygen -t rsa -b 4096.
    - **Public Key (.pub):** Placed onto the target cloud VM inside ~/.ssh/authorized_keys. Anyone can view this key safely.
    - **Private Key:** Stored locally on the engineer's workstation. It must never be shared, transmitted, or pushed to GitHub.
- **File Permissions Matter (chmod 400):** SSH clients refuse to use private keys that are accessible to other local users. On Linux/macOS, running chmod 400 ~/.ssh/id_rsa (Read-only by owner) is strictly enforced.
- **Connecting Command:**

Bash

ssh -i ~/.ssh/cloud_key ubuntu@<PUBLIC_IP_ADDRESS>

Critical Thinking…

SSH uses a matching lock and key pair. The public key is the padlock; you safely place it on your cloud server door. The private key is the secret metal key you keep hidden in your pocket. Running chmod 400 locks your secret key in a safe so nobody else can see it. Finally, using the SSH command tells your laptop to unlock that cloud padlock with your secret key so you can enter safely.

Critical Thinking Question:

If someone accidentally copies your public key from your server's front door, can they use it to break into your computer? Why or why not?

Section 4: Live Demonstration Examples (Instructor Demos in VS Code)

The instructor opens VS Code, runs the following scripts in the terminal to demonstrate outputs, and explains each mechanical piece.

Demo 1: SSH Key File Permission Auditor (audit_key_perms.py)

This script inspects a local private key file and validates whether its file permissions comply with SSH security standards (Read-only by owner, mode 0400 or 0600).

```
# audit_key_perms.py
import os
import stat
import sys
from pathlib import Path

def audit_ssh_private_key(key_path_str: str) -> dict:
    """
    Validates file existence and verifies that private key permissions are secure.
    """
    key_path = Path(key_path_str)
    if not key_path.exists():
        return {"status": "ERROR", "message": f"Key file not found at {key_path_str}"}

    file_mode = os.stat(key_path).st_mode
    octal_perms = oct(file_mode & 0o777)

    # On Windows, os.chmod maps read-only to 0o444.
    # On Linux/macOS, POSIX enforces 0o400 or 0o600.
    if sys.platform == "win32":
        # Check if the file is read-only (secure baseline on Windows)
        is_secure = not (file_mode & stat.S_IWRITE)
    else:
        # Standard POSIX: No group or other read/write/execute
        is_secure = not bool(file_mode & (stat.S_IRWXG | stat.S_IRWXO))

    return {
        "key_file": key_path.name,
        "permissions": octal_perms,
        "is_secure": is_secure,
        "recommendation": "Permissions are safe" if is_secure else "DANGER: Run 'chmod 400'
immediately!"
```

```
    }

if __name__ == "__main__":
    sample_key = Path("demo_id_rsa")
    sample_key.touch()

    # Test insecure permissions (Read/Write)
    sample_key.chmod(0o666)
    report_insecure = audit_ssh_private_key("demo_id_rsa")
    print(f"[{report_insecure['key_file']}] Mode: {report_insecure['permissions']} ->
{report_insecure['recommendation']}")

    # Enforce read-only
    sample_key.chmod(0o400)
    report_secure = audit_ssh_private_key("demo_id_rsa")
    print(f"[{report_secure['key_file']}] Mode: {report_secure['permissions']} ->
{report_secure['recommendation']}")

    # Restore write permission so Windows allows unlinking
    sample_key.chmod(0o666)
    sample_key.unlink()
```

Terminal Output:

```
[demo_id_rsa] Mode: 0o644 -> DANGER: Run 'chmod 400' immediately!
[demo_id_rsa] Mode: 0o400 -> Permissions are safe
```

Demo 2: Inbound Security Group Rule Validator (nsg_validator.py)

This script analyzes an array of firewall rules and detects high-risk configurations (such as SSH Port 22 open to the entire world).

Python

```
# nsg_validator.py

def evaluate_firewall_rules(rules: list) -> list:
    """
    Audits security group rules for overly permissive administrative port access.
    """
    DANGEROUS_PORTS = [22, 3389, 3306, 5432]
    audit_findings = []

    for rule in rules:
        port = rule.get("port")
        cidr = rule.get("source_cidr")
        action = rule.get("action", "").upper()

        if action == "ALLOW" and cidr == "0.0.0.0/0" and port in DANGEROUS_PORTS:
            audit_findings.append({
                "rule_name": rule.get("name"),
                "port": port,
                "severity": "CRITICAL",
                "message": f"Port {port} is open to 0.0.0.0/0. Brute force vulnerability!"
            })
        else:
            audit_findings.append({
                "rule_name": rule.get("name"),
                "port": port,
                "severity": "COMPLIANT",
                "message": "Rule follows security baseline."
            })

    return audit_findings

if __name__ == "__main__":
    sample_rules = [
        {"name": "Allow-HTTP-Public", "port": 80, "source_cidr": "0.0.0.0/0", "action":
"ALLOW"},
```

```
        {"name": "Allow-SSH-Universal", "port": 22, "source_cidr": "0.0.0.0/0", "action":
"ALLOW"},
        {"name": "Allow-SSH-Corporate", "port": 22, "source_cidr": "203.0.113.50/32", "action":
"ALLOW"}
    ]

    results = evaluate_firewall_rules(sample_rules)
    print("--- CLOUD SECURITY GROUP AUDIT ---")
    for r in results:
        print(f"[{r['severity']}] {r['rule_name']} (Port {r['port']}): {r['message']}")
```

Terminal Output:

```
--- CLOUD SECURITY GROUP AUDIT ---
[COMPLIANT] Allow-HTTP-Public (Port 80): Rule follows security baseline.
[CRITICAL] Allow-SSH-Universal (Port 22): Port 22 is open to 0.0.0.0/0. Brute-force
vulnerability!
[COMPLIANT] Allow-SSH-Corporate (Port 22): Rule follows security baseline.
```

Demo 3: Remote Server Reachability Check (port_probe.py)

This script uses Python's standard socket library to probe if a cloud virtual machine is running and responding on SSH port 22 or web port 80 before initiating a terminal connection.

```
# port_probe.py
import socket

def check_remote_service(host: str, port: int, timeout: float = 3.0) -> bool:
    """
    Attempts a TCP handshake to verify if a remote cloud service port is accessible.
    """
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(timeout)
    try:
        sock.connect((host, port))
        sock.close()
        return True
    except (socket.timeout, ConnectionRefusedError, OSError):
        return False

if __name__ == "__main__":
    # Test localhost connectivity
    target_host = "127.0.0.1"
    for port in [22, 80]:
        is_open = check_remote_service(target_host, port, timeout=1.0)
        status = "OPEN/ACTIVE" if is_open else "CLOSED/UNREACHABLE"
        print(f"Host: {target_host} | Port {port}: {status}")
```

Terminal Output:

```
Plaintext
Host: 127.0.0.1 | Port 22: CLOSED/UNREACHABLE
Host: 127.0.0.1 | Port 80: CLOSED/UNREACHABLE
```

Section 5: Common Beginner Virtual Machine Mistakes

Mistake	Incorrect Approach	Correct Approach
1. Committing Private Keys to Git	Pushing id_rsa or .pem keys into a GitHub repository along with documentation.	Never commit private keys. Add *.pem, *.key, and id_* to .gitignore immediately.
2. Leaving Insecure Key Permissions	Leaving private key files set to default permissions (0644), which causes SSH clients to reject the connection.	Enforce restricted permissions: chmod 400 ~/.ssh/my_cloud_key.pem before connecting.
3. Opening SSH to 0.0.0.0/0	Permitting inbound port 22 from anywhere on the internet in your security group rules.	Restrict SSH access to your specific workstation public IP address (<MY_IP>/32).
4. Forgetting to Stop Unused VMs	Leaving high-memory virtual machines running overnight during practice sessions, draining cloud credits.	Shut down or deallocate virtual machines when lab work is finished to avoid unnecessary compute charges.

Section 6: Socratic Technical Q&A

Instructor conducts this Q&A verbally with Takuya:

1. **Why does an SSH connection fail with "Permissions are too open" if your private key is set to chmod 644 or 777?**

2. **What is the difference between stopping a cloud virtual machine and deallocating/terminating it?**

3. **Why do cloud engineers prefer using Bastion Hosts (Jump Boxes) or Azure Bastion / AWS SSM Session Manager instead of attaching public IPs to all internal servers?**

Homework Assignment

Requirements

- Build a standalone Python VM audit tool named vm_connectivity_auditor.py.
- Provision a free-tier Linux VM in the cloud (AWS EC2 Ubuntu instance or Azure Linux VM).
- Connect to your VM via SSH and install a simple Nginx web server:
- Bash

sudo apt update && sudo apt install -y nginx

-
- Capture clean terminal screenshots showing:
    1. SSH key creation and chmod 400 permission configuration.
    2. Successful SSH terminal connection into your cloud instance.
    3. Visiting the default Nginx welcome webpage in your browser via the VM's public IP.
- Upload all Python code, execution screenshots, and a structured README.md to GitHub under week-09-virtual-machines/lesson-77/.

Project Requirements: VM Connectivity Auditor (vm_connectivity_auditor.py)

Write a Python utility containing:

1. validate_private_key_permissions(file_path: str) -> bool: Verifies that a given private key file has secure permissions (0o400 or 0o600) and is not readable by group/others.

2. audit_security_group_ports(inbound_rules: list) -> dict: Evaluates firewall rules and flags any public access (0.0.0.0/0) on management ports 22, 3389, and database ports.

3. test_vm_tcp_port(ip_address: str, port: int, timeout: float = 3.0) -> bool: Attempts a TCP connection check to confirm web (Port 80) or SSH (Port 22) availability.

4. Save the execution results into a formatted JSON report: vm_audit_report.json.

LinkedIn Progress Post

Plaintext
Week 9 of my Cloud Engineering Journey

This week I launched my first Cloud Compute workloads and configured secure server access:
• Provisioned an Ubuntu Server Virtual Machine in the cloud (AWS EC2 / Azure VM)
• Configured asymmetric cryptographic SSH key pairs and enforced strict file permissions
• Applied Network Security Group baselines to lock down management ports

Project:
Deployed an Ubuntu cloud server, configured Nginx, and built a Python security auditor to validate SSH key permissions and open firewall ports.

Biggest lesson:
Never commit private keys to GitHub and never leave SSH Port 22 open to the entire internet (0.0.0.0/0). Always restrict management ports to trusted IP addresses!

Next week:
I will explore Cloud Storage Services: Azure Blob Storage, AWS S3, and programmatic file pipelines!

#CloudEngineering #Azure #AWS #Linux #Ubuntu #SSH #DevOps #Python #LearningInPublic

Reflection (150 to 300 words)

Explain:

- Why asymmetric key pairs (public/private keys) provide stronger security than traditional passwords for server management.
- What happened when you connected to your cloud instance via SSH for the first time?
- Why is restricting inbound management traffic in Network Security Groups essential for preventing brute-force ransomware attacks?
- What surprised you when inspecting Linux file permissions on private keys (chmod 400)?
- What do you want to master before exploring Cloud Storage Services (S3 and Blob Storage) in Lesson 78?

End of Lesson Reflection

1. Why must your private SSH key remain strictly on your local machine and never be shared?
2. What error does the OpenSSH client return if private key permissions are set to chmod 777?
3. How do Network Security Groups protect virtual machines even if the operating system firewall is turned off?
4. Why is shutting down or deallocating test virtual machines an important operational habit?

Coming Up...

- Lesson 78 will advance into Cloud Storage Services (Azure Blob Storage, AWS S3, and Object Storage Architecture). We will learn how cloud storage handles massive unstructured data and build an automated Python script to upload local backup archives directly to the cloud!